



XIAMEN
UNIVERSITY

第四讲 MATLAB数值分析

温程璐 人工智能系



知识要点

- 4.1 基本的数据分析
- 4.2 矩阵函数
- 4.3 多项式运算
- 4.4 函数和数值积分
- 4.5 数据分析
- 4.6 稀疏矩阵



4.1 基本数据分析

函数	功 能
max	求各列最大值
min	求各列最小值
mean	求各列平均值
std	求各列标准差
median	求各列中间元素
sum	求各列元素和

注意：Matlab的基本数据处理功能是按列进行的。



4.2 矩阵函数

□ 矩阵的分析计算：

求矩阵的行列式、秩、逆矩阵、特征向量等；

□ 矩阵的各种分解：

(将一个大矩阵分解为多个简单矩阵的连乘)

如：三角分解、正交分解、奇异值分解等。

□ 矩阵的交集运算：

格式：`intersect(A,B)`

功能：返回值为向量A，B的公共部分。

□ 矩阵的并集运算：

格式：`union(A,B)`

功能：返回值为向量A，B的相交部分。

□ 线性方程求解



4.2.1 线性方程组的求解（应用矩阵函数）

线性方程组一般形式： $AX=B$

（ A 为 $n \times m$ 矩阵）

- (1) 当 $n=m$ 时，此方程称为“恰定”方程
- (2) 当 $n>m$ 时，此方程称为“超定”方程
- (3) 当 $n<m$ 时，此方程称为“欠定”方程



4.2.1 线性方程组的求解（应用矩阵函数）

1. 恰定方程组的解（有唯一的一组解）

线性方程组一般形式： $AX=B$ A 为 $n \times m$ 矩阵)

当 $n=m$ 时，此方程成为“恰定”方程

$$AX=B \rightarrow A^{-1}AX = A^{-1}B \rightarrow X = A^{-1}B = A \setminus B$$

有两种求解方法：(1) $X = \text{inv}(A) * B$ （速度较慢）

(2) $X = A \setminus B$ （速度快, 精度高）

例 $x_1 + 2x_2 = 8$

$$2x_1 + 3x_2 = 13$$

$$\underline{n=m=2}$$

$$\begin{bmatrix} 1 & 2 \\ 2 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 8 \\ 13 \end{bmatrix}$$

$$A = [1, 2; 2, 3];$$

$$B = [8; 13];$$

$$X = \text{inv}(A) * B$$

$$XX = A \setminus B$$

$$\begin{array}{l} X = \\ \\ 2 \\ 3 \\ \\ XX = \\ \\ 2 \\ 3 \end{array}$$



4.2.1 线性方程组的求解（应用矩阵函数）

2. 超定方程组的解（没有精确解）

线性方程组一般形式： $AX=B$

（ A 为 $n \times m$ 矩阵）

当 $n > m$ 时，此方程成为“超定”方程

$$AX=B$$

$$\rightarrow A'AX=A'B$$

$$\rightarrow X=(A'A)^{-1}A'B=\text{pinv}(A)*B \quad (\text{广义逆})$$

有两种求解方法：

(1) $X=\text{pinv}(A)*B$ %求解Moore-penrose伪逆，不存在逆矩阵的时候替代逆矩阵，用于求解没有唯一解或多个解。

(2) $X=A \setminus B$ （用最小乘方法找一个精确解）



4.2.1 线性方程组的求解（应用矩阵函数）

例： 求解线性方程组：

$$x_1 + 2x_2 = 1$$

$$2x_1 + 3x_2 = 2$$

$$3x_1 + 4x_2 = 5$$

$n=3, m=2, n>m$, 超定

```
A=[1,2;2,3;3,4];
```

```
B=[1;2;5];
```

```
X=pinv(A)*B
```

```
XX=A\B
```

$$\begin{bmatrix} 1 & 2 \\ 2 & 3 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 5 \end{bmatrix}$$

X =

3.3333

-1.3333

XX =

3.3333

-1.3333



4.2.1 线性方程组的求解（应用矩阵函数）

3. 欠定方程组的解（有无穷多个解）

线性方程组一般形式： $AX=B$ （A 为 $n \times m$ 矩阵）

当 $n < m$ 时，此方程成为“欠定”方程

有两种求解方法：

(1) $X = \text{pinv}(A) * B$ （具有最小长度或范数的解）

(2) $X = A \backslash B$ （具有最多零元素的解）

例 $x_1 + 2x_2 + 3x_3 = 1$

$$2x_1 + 3x_2 + 4x_3 = 2$$

$n=2, m=3, n < m$, 欠定

$$\begin{bmatrix} 1 & 2 & 3 \\ 2 & 3 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

```
A=[1,2,3;2,3,4];
```

```
B=[1;2];
```

```
X=pinv(A)*B
```

```
XX=A\B
```

X =

```
0.8333  
0.3333  
-0.1667
```

XX =

```
1  
0  
0
```



4.3 多项式运算

4.3.1 多项式的表示

4.3.2 多项式的运算

4.3.3 多项式的求解

4.3.4 多项式的拟合

4.3.5 多项式的插值



4.3.1 多项式的表示

一般形式：

$$f(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$$

用系数向量来表示： $p=[a_n \ a_{n-1} \ \dots \ a_1 \ a_0]$

```
% B(s)=3*s^2+6*s+9
```

```
% A(s)=2*s^3+4*s^2+6*s+8
```

```
B=[3 6 9]; %仅表达出来系数向量
```

```
A=[2 4 6 8]; %仅表达出来系数向量
```



4.3.2 多项式的运算

1. 多项式的加减

对应系数相加减，如果系数长度不等，应在前面补零。

例如： $p1=[1\ 2\ 3]$; $p2=[1\ 3\ 5]$; $p3=[1\ 3]$

则： $p1+p2=[2\ 5\ 8]$

$p1+p3=[1\ 3\ 6]$



4.3.2 多项式的运算

2. 多项式的乘法（数组卷积）

$$(ax^2 + bx + c) \square (dx + e) \\ = adx^3 + (ae + bd)x^2 + (be + cd)x + ce$$

<i>a</i>	<i>b</i>	<i>c</i>	
<i>d</i>	<i>e</i>		
<i>ad</i>	<i>bd</i>	<i>dc</i>	
	<i>ae</i>	<i>be</i>	<i>ec</i>
<i>ad</i>	<i>bd + ae</i>	<i>dc + be</i>	<i>ec</i>



4.3.2 多项式的运算

2. 多项式的乘法

格式: $\text{conv}(p_1, p_2)$ (卷积)

例如: $p_1=[1 \ 1];$

$p_2=[1 \ 2];$

$p_3=\text{conv}(p_1, p_2)=[1 \ 3 \ 2];$



4.3.2 多项式的运算

3. 多项式的除法

格式： $[q,r]=\text{deconv}(p1,p2)$ (q 商， r 余数)

例如： $p1=[1\ 1];$

$p3=[1\ 3\ 2];$

$[q\ ,r]=\text{deconv}(p3,p1)$



4.3.3 多项式的求解

1. 多项式的求导与积分（微分）

格式：polyder(p)

例如：p=[1 2 3 4];

$$f(x) = x^3 + 2x^2 + 3x + 4$$

polyder(p)的运算结果为[3 4 3]

格式：polyint(P)

$$f'(x) = 3x^2 + 4x + 3$$



4.3.3 多项式的求解

2. 多项式的求根

■ 格式: `roots(p)` (由多项式求根)

例如: `p=[1 3 2];`

`roots(p)`的运算结果为`[-2 ; -1 ]`

$$x^2 + 3x + 2 = (x + 1)(x + 2)$$

■ 格式: `poly(r)` (由根求多项式)

当`r`为向量时, `poly` 把`r`作为根求出多项式。

如: `r=[-2; -1]`, `poly(r)`的运算结果为`[1 3 2]`

当`r`为方阵时, `poly(r)` 即为方阵`r`的特征多项式



4.3.3 多项式的求解

3. 多项式的求值

格式： $y = \text{polyval}(p, x)$

(计算多项式 p 在 x 的每个点处的数值)

$y = \text{polyvalm}$ 以矩阵方式计算多项式

例如： $p = [1 \ 2 \ 3];$

$\text{polyval}(p, 1)$ 的运算结果为6

4、分式展开

$[r, p, k] = \text{residue}(a, b);$



课堂练习：

$$H(s) = \frac{3s^2 + 6s + 9}{2s^3 + 4s^2 + 6s + 8}$$

求出该系统的频率响应并画出频率特性？



例题

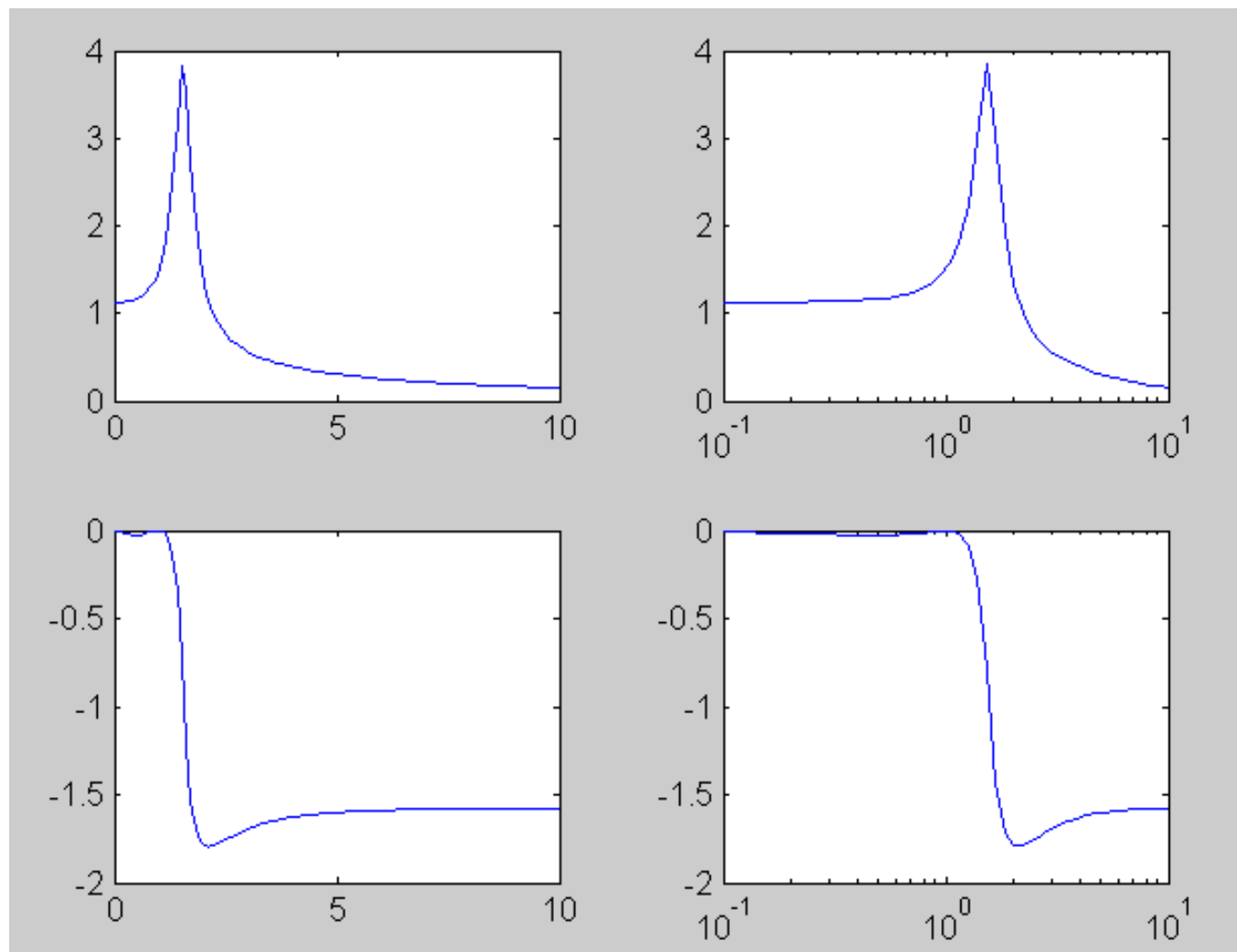
```
lc;
clear all;
%多项式求值的应用
%B(s)=3*s^2+6*s+9
%A(s)=2*s^3+4*s^2+6*s+8
%H(s)=B(s)/A(s)
B=[3 6 9];A=[2 4 6 8];
w=linspace(0,10);
BB=polyval(B, j*w); %多项式求值
AA=polyval(A, j*w); %多项式求值
subplot(2,2,1);
plot(w,abs(BB ./ AA));
subplot(2,2,3);
plot(w,angle(BB ./ AA));
```

```
w1=logspace(-1,1);
B1=polyval(B,j*w1); %多项式求值
A1=polyval(A,j*w1); %多项式求值
subplot(2,2,2);
semilogx(w1,abs(B1./A1));
subplot(2,2,4);
semilogx(w1,angle(B1./A1));
```

Semilogx使用 x 轴的以 10 为基数的对数刻度和 y 轴的线性刻度创建一个绘图。它绘制 Y 的列对其索引的图。 Y 的值可以是数值、日期时间、持续时间或分类值。



例题



4.3.4 多项式的拟合

多项式的拟合就是用多项式函数所表示的曲线来描述一些已知的点，使这些点尽量逼近曲线。

格式：`p=polyfit(x,y,n)`

`p = polyfit(x,y,n)` 返回多项式的幂次次数为 n 的多项式 $p(x)$ 的系数，该阶数是 y 中数据的最佳拟合（最小二乘）， x, y 为已知的点坐标向量。 p 中的系数按降幂排列， p 的长度为 $n+1$ 。

`[p,S] = polyfit(x,y,n)` 还返回一个结构体 S ，可作 `polyval` 的输入来获取误差估计值。

`[p,S,mu] = polyfit(x,y,n)` 还返回 μ ，后者是一个二元素向量，包含中心化值和缩放值。 $\mu(1)$ 是 `mean(x)`， $\mu(2)$ 是 `std(x)`。`polyfit` 将 x 的中心置于零值处并缩放为具有单位标准差。这种中心化和缩放变换可改善多项式和拟合算法的数值属性。



4.3.4 多项式的拟合

%在区间 $[0, 4\pi]$ 中沿正弦曲线生成 10 个等间距的点。

```
x = linspace(0,4*pi,10); y = sin(x);
```

%使用 `polyfit` 将一个 7 次多项式与这些点拟合。

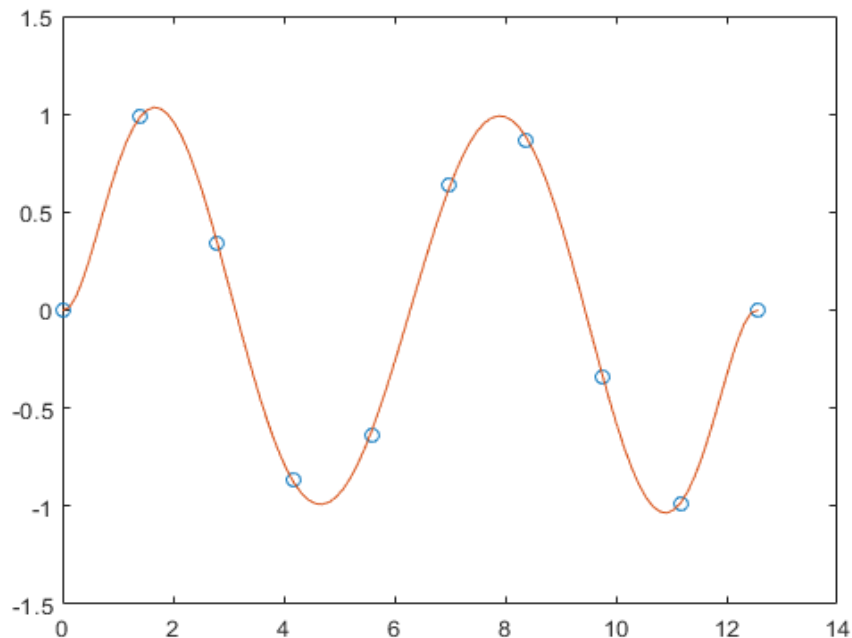
```
p = polyfit(x,y,7);
```

%在更精细的网格上计算多项式并绘制结果图。

```
x1 = linspace(0,4*pi);
```

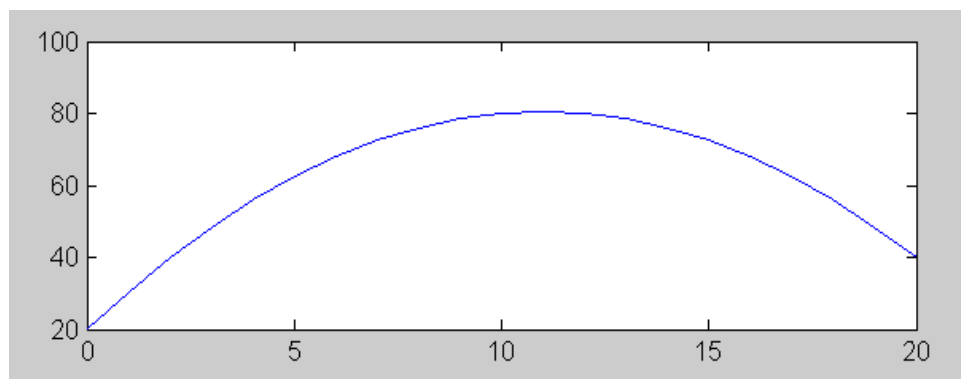
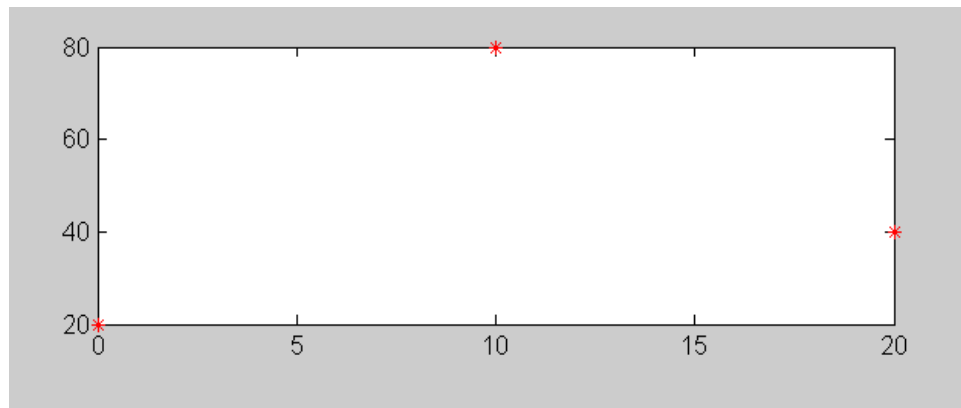
```
y1 = polyval(p,x1);
```

```
figure plot(x,y,'o') hold on plot(x1,y1)  
hold off
```



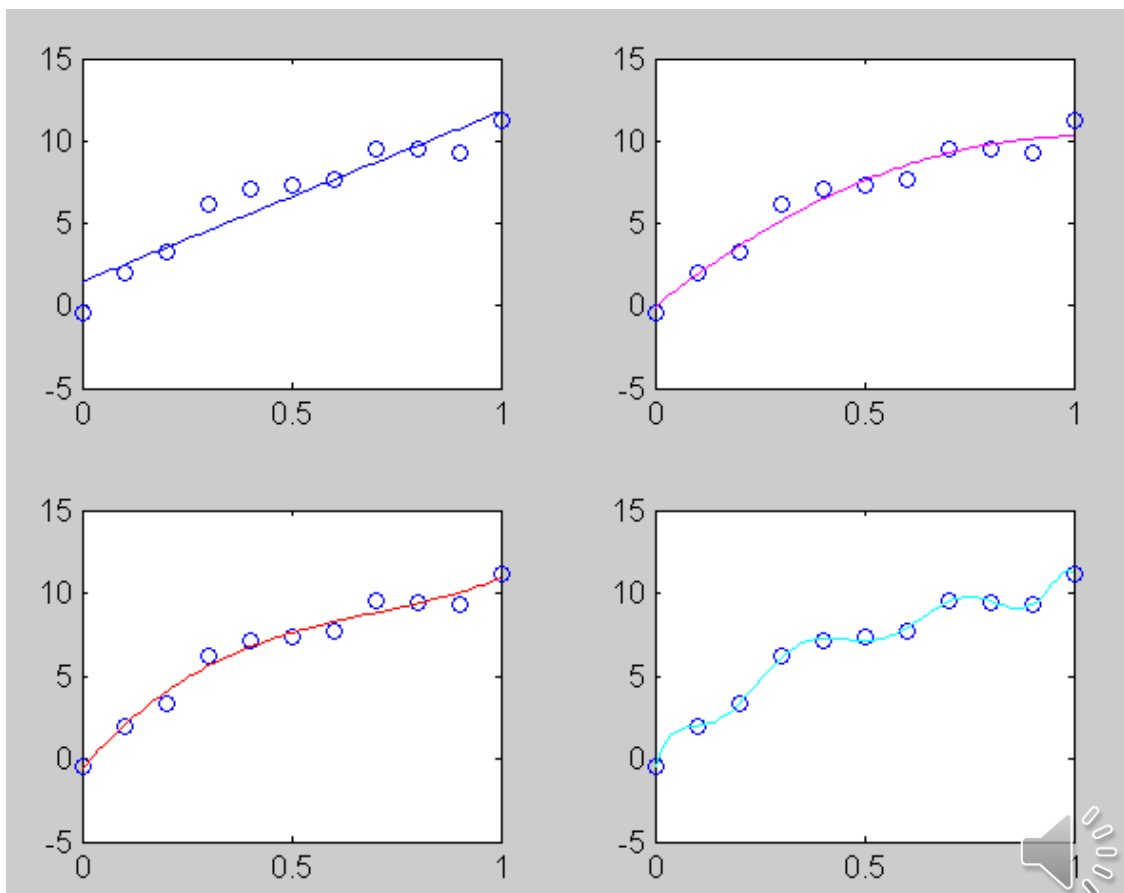
4.3.4 多项式的拟合

```
x=[0 10 20];  
y=[20 80 40];  
subplot(2,1,1);  
plot(x,y,'*r');  
%用 polyfit 将一个 2次多项式  
与这些点拟合。  
p=polyfit(x,y,2);  
subplot(2,1,2);  
plot((0:20)  
polyval(p,(0:20)));
```



例题

```
42 %多项式拟合二
43 x=0:0.1:1;
44 y=[-0.447, 1.978, 3.28, 6.16, 7.08, 7.34, 7.66, 9.56, 9.48, 9.30, 11.2];
45
46 a1=polyfit(x, y, 1);
47 x1=linspace(0, 1);
48 y1=polyval(a1, x1)
49 figure;
50 subplot(2, 2, 1);
51 plot(x, y, 'o', x1, y1, 'b');
52 a2=polyfit(x, y, 2);
53 y2=polyval(a2, x1);
54 subplot(2, 2, 2);
55 plot(x, y, 'o', x1, y2, 'm');
56 a3=polyfit(x, y, 3);
57 y3=polyval(a3, x1);
58 subplot(2, 2, 3);
59 plot(x, y, 'o', x1, y3, 'r');
60 a9=polyfit(x, y, 9);
61 y9=polyval(a9, x1);
62 subplot(2, 2, 4);
63 plot(x, y, 'o', x1, y9, 'c');
```



4.3.5 多项式的插值

插值是在一些已知点之间插入一些点，使这些点的连线与已知点连线更逼近。

1. 一维插值
2. 二维插值



4.3.5 多项式的插值

1、一维插值（平面插值）

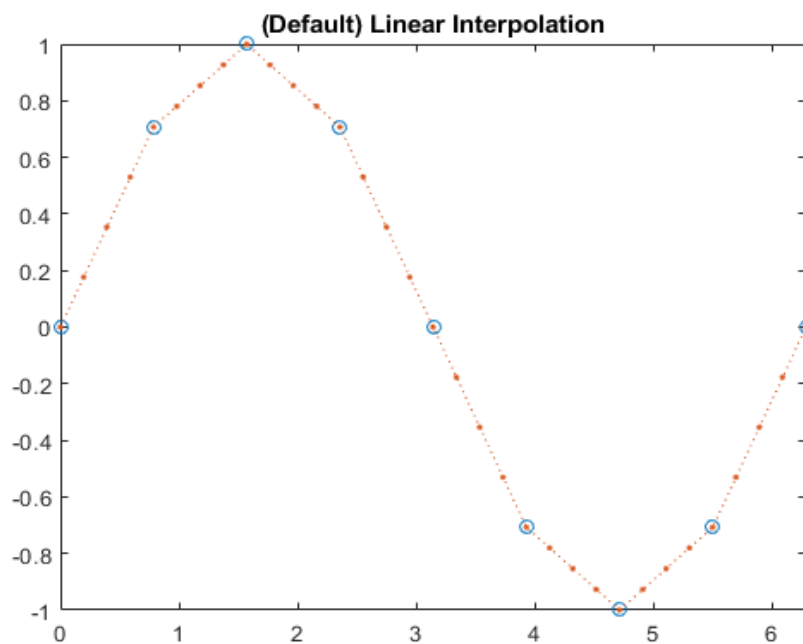
格式：

- $vq = \text{interp1}(x,v,xq)$ 使用线性插值返回一维函数在特定查询点的插入值。向量 x 包含样本点， v 包含对应值 $v(x)$ 。向量 xq 包含查询点的坐标。
- $vq = \text{interp1}(x,v,xq,\text{method})$ 指定备选插值方法：'linear'、'nearest'、'next'、'previous'、'pchip'、'cubic'、'v5cubic'、'makima' 或 'spline'。默认方法为 'linear'。
- $vq = \text{interp1}(x,v,xq,\text{method},\text{extrapolation})$ 用于指定外插策略，来计算落在 x 域范围外的点。
- $pp = \text{interp1}(x,v,\text{method},\text{'pp'})$ 使用 method 算法返回分段多项式形式的 $v(x)$ 。



例题

```
%定义样本点 x 及其对应样本值 v。  
x = 0:pi/4:2*pi;  
v = sin(x);  
%将查询点定义为 x 范围内更精细的  
采样点。  
xq = 0:pi/16:2*pi;  
%在查询点插入函数并绘制结果。  
figure vq1 = interp1(x,v,xq);  
plot(x,v,'o',xq,vq1,':');  
xlim([0 2*pi]);  
title('(Default) Linear Interpolation');
```

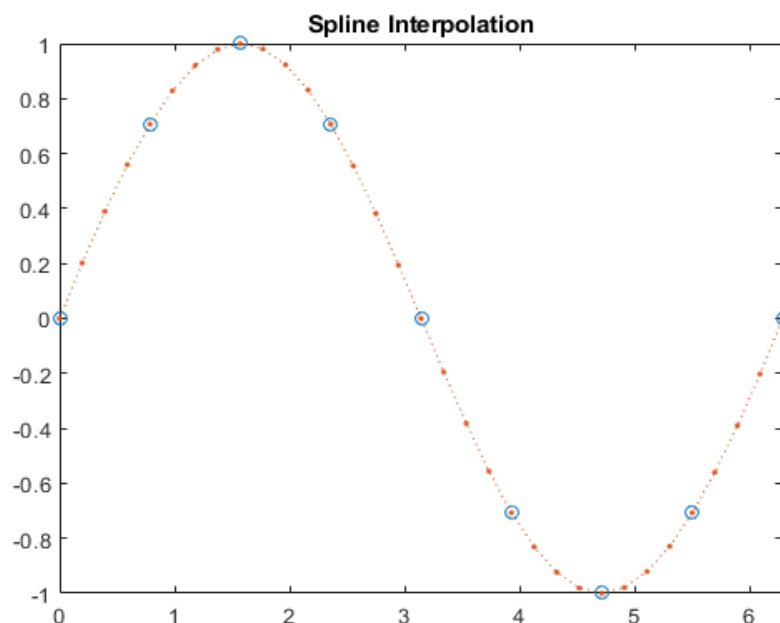


`xlim(limits)` 设置当前坐标区或图的 x 坐标轴范围。将 `limits` 指定为 `[xmin xmax]` 形式的二元素向量，其中 `xmax` 大于 `xmin`。



例题

```
%定义样本点 x 及其对应样本值 v。  
x = 0:pi/4:2*pi;  
v = sin(x);  
%将查询点定义为 x 范围内更精细的  
采样点。  
xq = 0:pi/16:2*pi;  
%在查询点插入函数并绘制结果。  
figure vq2 = interp1(x,v,xq,'spline');  
plot(x,v,'o',xq,vq2,':');  
xlim([0 2*pi]);  
title('Spline Interpolation');
```



4.3.5 多项式的插值

2、二维插值（立体）

格式：

- $V_q = \text{interp2}(X,Y,V,X_q,Y_q)$ 使用线性插值返回双变量函数在特定查询点的插入值。X 和 Y 包含样本点的坐标。V 包含各样本点处的对应函数值。X_q 和 Y_q 包含查询点的坐标。
- $V_q = \text{interp2}(V,X_q,Y_q)$ 假定一个默认的样本点网格。默认网格点覆盖矩形区域 $X=1:n$ 和 $Y=1:m$ ，其中 $[m,n] = \text{size}(V)$ 。
- $V_q = \text{interp2}(V)$ 将每个维度上样本值之间的间隔分割一次，形成优化网格，并在这些网格上返回插入值。
- $V_q = \text{interp2}(V,k)$ 将每个维度上样本值之间的间隔反复分割 k 次，形成优化网格，并在这些网格上返回插入值。这将在样本值之间生成 2^{k-1} 个插入点。
- $V_q = \text{interp2}(_,_,\text{method})$ 指定备选插值方法：'linear'、'nearest'、'cubic'、'makima' 或 'spline'。默认方法为 'linear'。



```
%对 peaks 函数进行粗略采样。
```

```
[X,Y] = meshgrid(-3:3);
```

```
V = peaks(X,Y);
```

```
%绘制粗略采样。
```

```
figure
```

```
surf(X,Y,V)
```

```
title('Original Sampling' );
```

```
%创建间距为 0.25 的查询网格。
```

```
[Xq,Yq] = meshgrid(-3:0.25:3);
```

```
%在查询点处插入值。
```

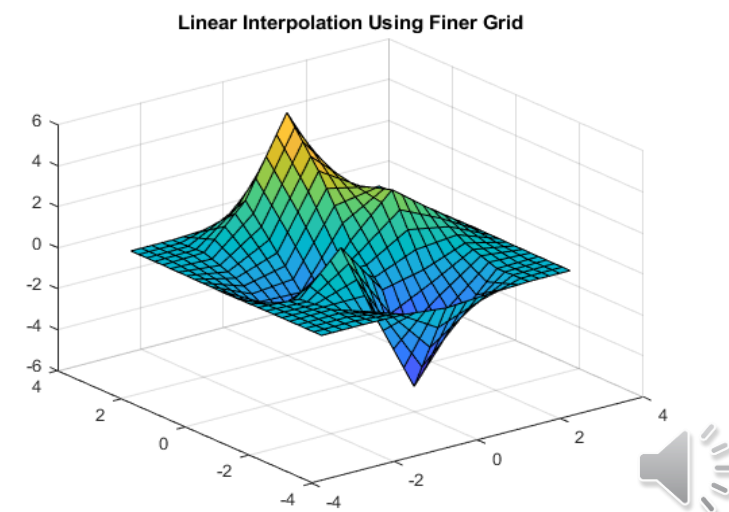
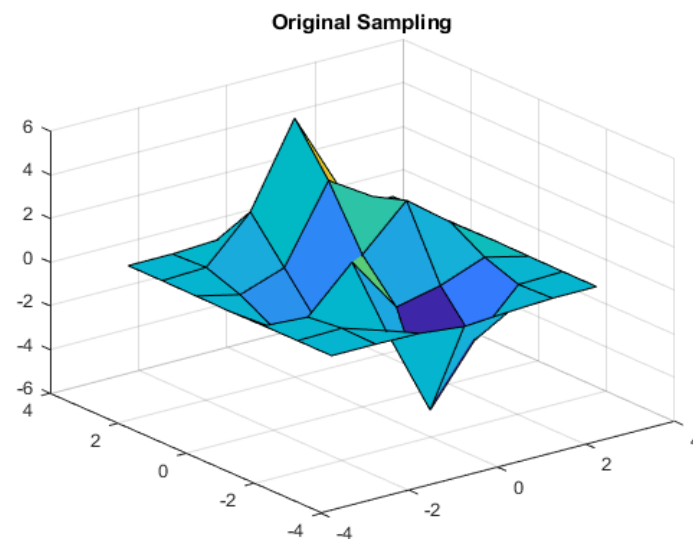
```
Vq = interp2(X,Y,V,Xq,Yq);
```

```
%绘制结果。
```

```
figure
```

```
surf(Xq,Yq,Vq);
```

```
title('Linear Interpolation Using Finer Grid');
```



例题

%对 peaks 函数进行粗略采样。

```
[X,Y] = meshgrid(-3:3);
```

```
V = peaks(X,Y);
```

%绘制粗略采样。

```
figure
```

```
surf(X,Y,V)
```

```
title('Original Sampling' );
```

%创建间距为 0.25 的查询网格。

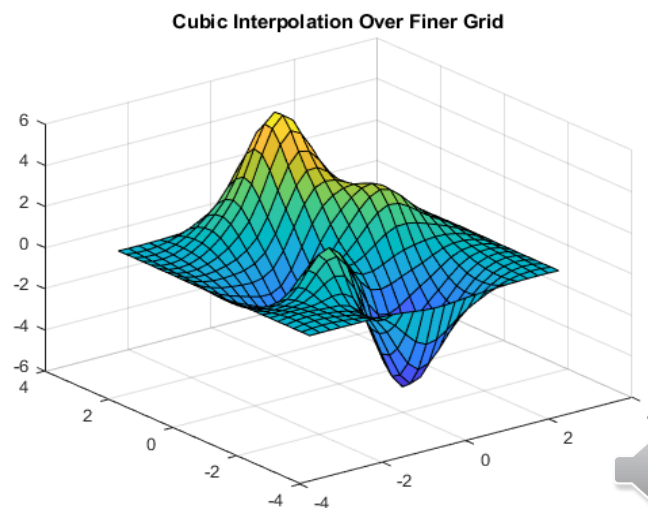
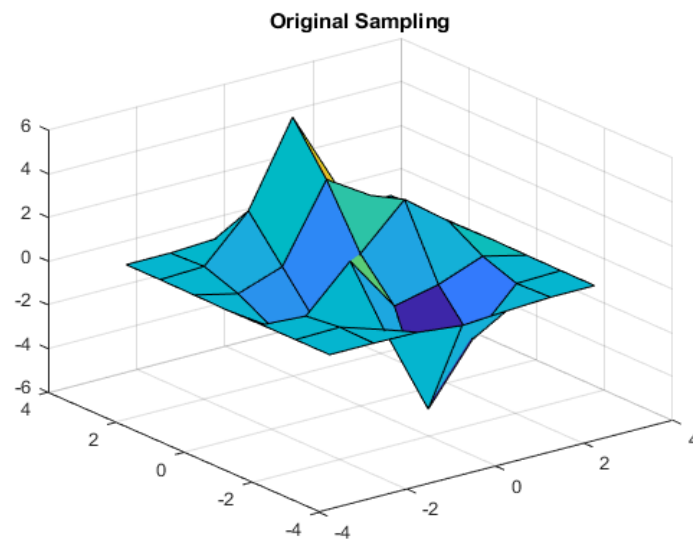
```
[Xq,Yq] = meshgrid(-3:0.25:3);
```

%在查询点处插入值，并指定三次插值

```
Vq = interp2(X,Y,V,Xq,Yq,'cubic' );
```

%绘制结果。

```
figure surf(Xq,Yq,Vq); title('Cubic  
Interpolation Over Finer Grid');
```



4.4 函数和函数积分

4.4.1 函数的绘图及分析

4.4.2 特殊函数

4.4.3 函数的数值积分

4.4.4 常微分方程的求解



4.4.1 函数的绘图及分析

- 函数和数值积分库 (funfun)
- 特殊函数库 (specfun)

1、绘制函数曲线

格式：

- `fplot(f)` 在默认区间 $[-5\ 5]$ (对于 x) 绘制由函数 $y = f(x)$ 定义的曲线
- `fplot(f,xinterval)` 将在指定区间绘图。将区间指定为 $[xmin\ xmax]$ 形式的二元素向量
- `fplot(funx,funy)` 在默认区间 $[-5\ 5]$ (对于 t) 绘制由 $x = funx(t)$ 和 $y = funy(t)$ 定义的曲线
- `fplot(funx,funy,tinterval)` 将在指定区间绘图。将区间指定为 $[tmin\ tmax]$ 形式的二元素向量
- `fplot(____,LineStyle)` 指定线型、标记符号和线条颜色。例如, `'-r'` 绘制一根红色线条。在前面语法中的任何输入参数组合后使用此选项
- `fplot(____,Name,Value)` 使用一个或多个名称-值对组参数指定线条属性。例如, `'LineWidth',2` 指定 2 磅的线宽



示例

%函数的绘图

subplot(1,3,1); % 一行三列小图

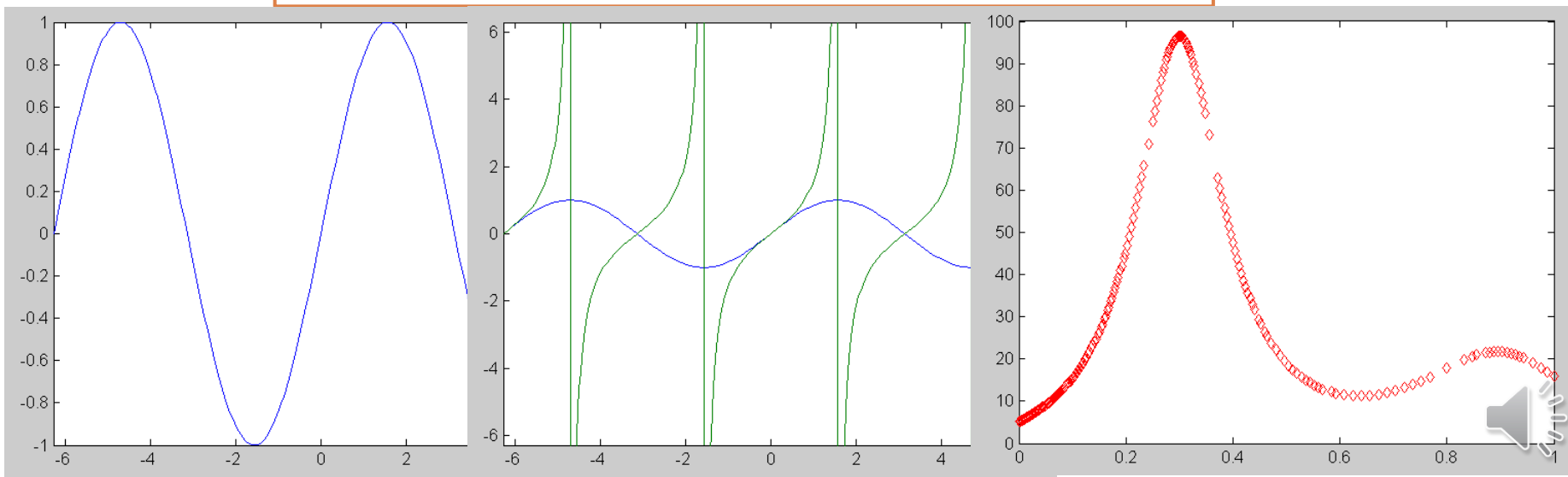
fplot('sin',2*pi*[-1 1]); %函数制图

subplot(1,3,2);

fplot('[sin(x) tan(x)]',2*pi*[-1 1 -1 1]);

subplot(1,3,3);

fplot('humps', [0 1], 'rd');



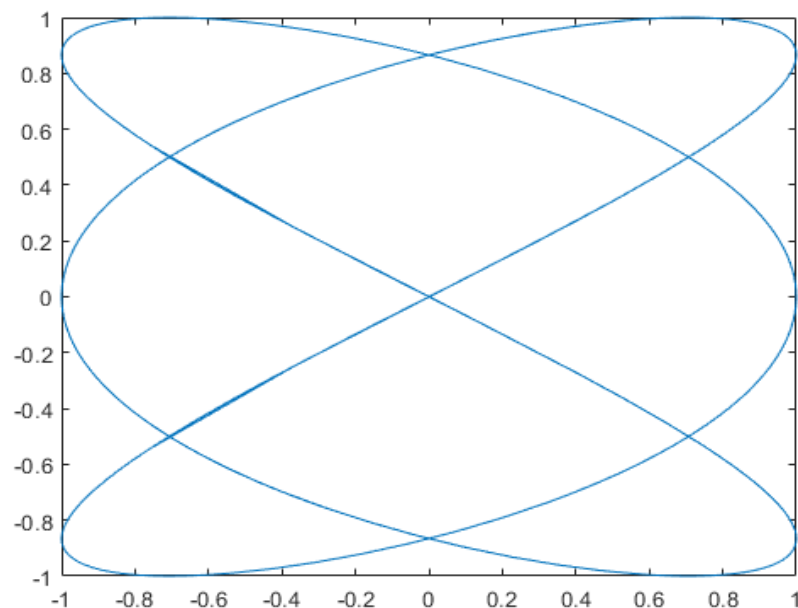
示例

%绘制参数化曲线 $x=\cos(3t)$ 和 $y=\sin(2t)$ 。

`xt = @(t) cos(3*t);`

`yt = @(t) sin(2*t);`

`fplot(xt,yt)`



4.4.2 函数的数值积分

1、定积分（一维数值积分）

格式：quad(‘函数名’, x_1 , x_2) （不推荐）
（或 quad8，高阶方法）

对函数在区间 $[x_1, x_2]$ 内的定积分

例如：quad(‘humps’,0,1)=29.8583 %求这个函数在[0,1]的定积分。

推荐函数：

- $q = \text{integral}(\text{fun}, \text{xmin}, \text{xmax})$ 使用全局自适应积分和默认误差容限在 xmin 至 xmax 间以数值形式为函数 fun 求积分。
- $q = \text{integral}(\text{fun}, \text{xmin}, \text{xmax}, \text{Name}, \text{Value})$ 指定具有一个或多个 Name, Value 对组参数的其他选项。



4.4.2 函数的数值积分

1、定积分（一维数值积分）

```
%创建函数  $f(x)=e^{-x^2}(\ln x)^2$ 。  
fun = @(x) exp(-x.^2).*log(x).^2;  
  
%计算  $x=0$  至  $x=\text{Inf}$  的广义积分。  
q = integral(fun,0,Inf)  
q = 1.9475
```

参数化函数

```
%创建带有一个参数  $c$  的函数  $f(x)=1/(x^3-2x-c)$ 。  
fun = @(x,c) 1./(x.^3-2*x-c);  
  
%在  $c=5$  时，计算从  $x=0$  至  $x=2$  的积分。  
q = integral(@(x)fun(x,5),0,2)  
q = -0.4605
```



4.4.2 函数的数值积分

2、二重数值积分

格式：

$q = \text{integral2}(\text{fun}, \text{xmin}, \text{xmax}, \text{ymin}, \text{ymax})$ ：在平面区域 $\text{xmin} \leq x \leq \text{xmax}$ 和 $\text{ymin}(x) \leq y \leq \text{ymax}(x)$ 上逼近函数 $z = \text{fun}(x, y)$ 的积分。

$q = \text{integral2}(\text{fun}, \text{xmin}, \text{xmax}, \text{ymin}, \text{ymax}, \text{Name}, \text{Value})$ ：指定具有一个或多个 Name, Value 对组参数的其他选项。



4.4.3 常微分方程的求解

(ODE , Ordinary Differential Equation)

格式： $[x,y]=\text{ode23}(\text{'函数名'}, x_0, x_n, y_0)$ (2,3阶)

或 ode45 (4,5阶)

函数名为微分方程名（含 x,y ）， x_0, x_n 为 x 区间， y_0 为 y 初值。

- $[t,y] = \text{ode23}(\text{odefun}, \text{tspan}, y_0)$ （其中 $\text{tspan} = [t_0 \text{ } t_f]$ ）求微分方程组 $y' = f(t,y)$ 从 t_0 到 t_f 的积分，初始条件为 y_0 。解数组 y 中的每一行都与列向量 t 中返回的值相对应。
- 所有 MATLAB® ODE 求解器都可以解算 $y' = f(t,y)$ 形式的方程组。
 ode23s 求解器只能解算质量矩阵为常量的问题。 ode15s 和 ode23t 可以解算具有奇异质量矩阵的问题，称为微分代数方程 (DAE)。
- $[t,y] = \text{ode23}(\text{odefun}, \text{tspan}, y_0, \text{options})$ 还使用由 options （使用 odeset 函数创建的参数）定义的积分设置。例如，使用 AbsTol 和 RelTol 选项指定绝对误差容限和相对误差容限，或者使用 Mass 选项提供质量矩阵。



4.4.3 常微分方程的求解

例如：求微分方程：

$$dy/dx = -3y + 2x, \quad y(1) = 2 \quad \text{区间为 } [1, 3] \text{ 的解}$$

首先建立函数：f=myde(x,y) (myde.m)

$$f = -3*y + 2*x$$

$$[x,y] = \text{ode23}('myde', 1, 3, 2)$$



4.5 数据分析和傅里叶变换

4.5.1 数据的基本分析

4.5.2 特殊函数

4.5.3 函数的数值积分



4.5.1 数据的基本分析（按列分析）

1、求最大最小值： $\max(\text{data}) / \min(\text{data})$

例如： $a = [1\ 2\ 3; 2\ 3\ 4; 4\ 5\ 6]$; $\max(a) = [4\ 5\ 6]$

2、求平均值： $\text{mean}(\text{data})$

例如： $a = [1\ 2\ 3; 2\ 3\ 4; 4\ 5\ 6]$; $\text{mean}(a) = [2.33\ 3.33\ 4.33]$

3、求和： $\text{sum}(\text{data})$

例如： $a = [1\ 2\ 3; 2\ 3\ 4; 4\ 5\ 6]$; $\text{sum}(a) = [7\ 10\ 13]$

4、差分： $\text{diff}(\text{data})$ (后行元素-前行元素值)

例如： $a = [1\ 2\ 3; 2\ 3\ 4; 4\ 5\ 6]$; $\text{diff}(a) = [1\ 1\ 1; 2\ 2\ 2]$



4.5.2 相关与卷积

对两组数据（或两个信号），可求其相关、协方差和卷积等

1、求协方差：

$\text{cov}(x)$ 求 x 的协方差阵

$\text{cov}(x,y)$ 求 x,y 的协方差

2、求相关系数

$\text{corrcoef}(x)$ 求 x 的自相关阵

$\text{corrcoef}(x,y)$ 求 x,y 的互相关系数

3、求卷积

$\text{conv}(x,y)$



4.5.2 傅里叶变换

可对数据（离散信号）求傅立叶变换

1、离散傅立叶变换：

$\text{fft}(x)$ 对序列 x 求DFT

$\text{fft}(x,N)$ 对序列 x 求FFT

例如： $x=[1\ 2\ 3\ 4]$

$$y=\text{fft}(x)=[10\ -2+2i\ -2\ -2-2i]$$

2、离散傅立叶反变换：

$\text{ifft}()$ 同上。



4.6 稀疏矩阵

4.6.1 稀疏矩阵的创建

4.6.2 稀疏矩阵的运算



4.6.1 稀疏矩阵的创建

稀疏矩阵函数库 (sparfun)

工程中会遇到很大的矩阵，其元素大多数为0。描述这类矩阵时，为了节省空间提高速度，只存储非0元素，这种矩阵称为稀疏矩阵。

而一般的矩阵称为完全矩阵。

稀疏矩阵有自己的理论和方法。



4.6.1 稀疏矩阵的创建

1、由完全矩阵得到

格式：`s=sparse(x)` (`x`是完全矩阵)

例如：`x=[1 0 0 2 ;3 0 3 0 ];`

`s=sparse(x)`



例题

```
5 %稀疏矩阵的创建
6 x=[0 0 0 0 0;
7     0 0 0 0.1302 0;
8     -0.5963 0 0 0 0;
9     0 1.4499 0.0388 0 0;
10    0.5589 0 0 0 -0.0954];
11 x
12 s=sparse(x);
13 s
14 whos
```

```
x =

     0     0     0     0     0
     0     0     0    0.1302     0
   -0.5963     0     0     0     0
     0    1.4499    0.0388     0     0
    0.5589     0     0     0   -0.0954
```

s =

```
(3,1)   -0.5963
(5,1)    0.5589
(4,2)    1.4499
(4,3)    0.0388
(2,4)    0.1302
(5,5)   -0.0954
```

Name	Size	Bytes	Class
s	5x5	96	double array (sparse)
x	5x5	200	double array

4.6.1 稀疏矩阵的创建

2、直接创建

格式： `s=sparse(i,j,v,m,n)`

`i,j`为非0元素行下标和列下标向量

`v`是非0元素值向量

`m,n`是待生成稀疏矩阵的行数和列数



4.6.1 稀疏矩阵的创建

例如：

```
s=sparse([1 1 2 2],[1 4 1 3],[1 2 3 3],2,4);
```

(1,1) 1

(2,1) 3

(2,3) 3

(1,4) 2



4.6.2 稀疏矩阵的运算

1、利用初等运算的命令

(如：四则运算，求逆等)

例如： $a=[1\ 0\ 0\ 2]$; $b=[0\ 2\ 0\ 3]$;

$sa=sparse(a)$; $sb=sparse(b)$;

$(sa+sb)=?$

2、必要时，转为完全矩阵，

格式： $full(s)$



本章节结束，谢谢



1. 角度 $x = [30 \ 45 \ 60]$ ，求 x 的正弦、余弦、正切和余切。

2. 将矩阵 $a = \begin{bmatrix} 4 & 2 \\ 7 & 5 \end{bmatrix}$ 、 $b = \begin{bmatrix} 7 & 1 \\ 8 & 3 \end{bmatrix}$ 和 $c = \begin{bmatrix} 5 & 9 \\ 6 & 2 \end{bmatrix}$ 组合成两个新矩阵：

(1) 组合成一个 4×3 的矩阵，第一列为按列顺序排列的 a 矩阵元素，第二列为按列顺序排列的 b 矩阵元素，第三列为按列顺序排列的 c 矩阵元素，即 $\begin{bmatrix} 4 & 7 & 5 \\ 7 & 8 & 6 \\ 2 & 1 & 9 \\ 5 & 3 & 2 \end{bmatrix}$ 。

(2) 按照 a 、 b 、 c 的列顺序组合成一个行矢量，即

$[4 \ 5 \ 2 \ 7 \ 7 \ 8 \ 1 \ 3 \ 5 \ 6 \ 9 \ 2]$ 。

3. 用 MATLAB 语句输入矩阵 A : $A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 4 & 3 & 2 & 1 \\ 2 & 3 & 4 & 1 \\ 3 & 2 & 4 & 1 \end{bmatrix}$ 。

(1) $A(10)$ 的值是多少；(2) $A(6)-A(4,2)=?$ (3) 给出 $A(5,6)=5$ 命令将得出什么结果？

4. 求解多项式 $x^4-7x^2+2x+40$ 的根。

5. 已知有一组数据如下：

X 0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1

Y -0.44 1.97 3.28 6.16 7.08 7.34 7.66 9.56 9.48 9.3 11.2

分别用 3 次和 6 次多项式拟合这些数据，写出 Matlab 语句。